

AgroVision

Sistema de Identificacao de Plantas Agricolas
via Inteligencia Artificial Embarcada

Documentacao Tecnica Completa para TCC

Plataforma:	React Native / Expo (iOS & Android)
IA Embarcada:	TensorFlow Lite - MobileNetV2 INT8
Pipeline ML:	PlantNet300k + PlantVillage CLAHE + Transfer Learning
Banco de Dados:	SQLite local (expo-sqlite)
Arquitetura:	Offline-First Sem dependencia de servidor

1. Introducao e Contexto do Projeto

O AgroVision e um aplicativo mobile desenvolvido para auxiliar agricultores e profissionais agronomos na identificacao de especies vegetais de cultivo agricola. Utilizando visao computacional embarcada diretamente no dispositivo movel, o sistema realiza inferencia offline com um modelo de deep learning quantizado (TFLite), sem necessidade de conexao com servidores externos.

A proposta central do projeto e tornar a tecnologia de reconhecimento de imagem acessivel ao campo, onde a conectividade de internet e frequentemente limitada ou inexistente. O modelo e treinado com datasets publicos de alta qualidade (PlantNet300k e PlantVillage), preprocessados com tecnicas de normalizacao de iluminacao (CLAHE) para robustez em condicoes reais de campo.

1.1 Problema

- Agricultores nao possuem acesso facil a identificacao precisa de especies vegetais
- Ferramentas existentes exigem conexao constante com internet
- Modelos generico de IA nao sao otimizados para cultivos agricolas brasileiros
- Falta de feedback agronomico pratico apos a identificacao

1.2 Solucao Proposta

- App mobile 100% offline com modelo TFLite embarcado (~12 MB)
- Identificacao de 30+ especies agricolas com 85-90% de acuracia estimada
- Recomendacoes agronomicas em portugues para cada especie identificada
- Historico local de analises via SQLite
- Biblioteca institucional com conteudo agronomico

1.3 Objetivos do TCC

- Desenvolver pipeline completo de treinamento de modelo de visao computacional
- Implementar arquitetura offline-first em aplicativo React Native
- Aplicar Transfer Learning com MobileNetV2 para dominio agricola
- Validar conceito cientifico de Explainability com Grad-CAM (intervencao TCC)
- Documentar processo reproduzivel de preparacao de dados e treinamento

2. Arquitetura do Sistema

2.1 Visao Geral - Arquitetura Offline-First

O sistema e dividido em dois modulos independentes que interagem apenas no momento do deploy do modelo treinado:

```
App Mobile (React Native/Expo)
+-- TFLite Inference (100% offline)
|   +-- model_species.tflite (~12MB INT8 quantizado)
|   +-- labels_species.json (mapeamento de classes)
|
+-- SQLite Database (historico local)
|
+-- Aba Institucional (biblioteca agronomica)

Pipeline de Treinamento (Python - Standalone)
+-- HuggingFace: PlantNet300k + PlantVillage
+-- prepare_species.py: CLAHE + Padding Proporcional
+-- train_species.py: MobileNetV2 + Fine-tuning + INT8
    --> model_species.tflite
```

2.2 Fluxo de Dados do Aplicativo

```
Usuario (Camera ou Galeria)
|
v
CameraScreen <-- cameraService (permissoes + captura)
|
v
ProcessingScreen (indicador de carregamento)
|
v
useAnalysis Hook
|
v
AnalysisContext.analyze()
|--- analysisService.analyzeImage()
|     |--- tfliteService.predict() <-- TFLite model
|     +--- Fallback: mock response
|
|--- databaseService.saveAnalysis() <-- SQLite INSERT
|
+--- Update Context State
|
v
ResultScreen (especie, confianca, recomendacoes)
```

|
v

Historico salvo localmente (SQLite)

2.3 Decisoes Arquiteturais Chave

Offline-First:	TFLite roda 100% no dispositivo, sem servidor externo
Servicos Modulares:	API, Database, Camera e ML isolados em servicos independentes
Context API + Reducer:	Gerenciamento de estado centralizado sem Redux
TypeScript Full:	Tipagem completa em todo o frontend React Native
Transfer Learning:	MobileNetV2 pre-treinado no ImageNet, fine-tuned para plantas
Augmentacao Training-Only:	Layers de augmentacao fora do modelo = TFLite compative
Treinamento em 2 Fases:	Base congelada -> fine-tuning incremental para melhor acuracia
CLAHE Preprocessing:	Normalizacao de iluminacao para fotos de campo
Padding Proporcional:	Preserva aspect ratio sem distorcer caracteristicas da planta
Quantizacao INT8:	Modelo ~12 MB, inferencia <500ms no dispositivo

3. Stack Tecnologica Completa

3.1 Frontend - React Native / Expo

Biblioteca	Versao	Funcao
React Native	0.81.5	Framework de desenvolvimento mobile cross-platform
Expo SDK	54.0.0	Ferramentas, build e acesso a APIs nativas
TypeScript	5.3.0	Tipagem estatica para maior confiabilidade
React Navigation	latest	Navegacao por abas e pilhas de telas
react-native-tflite	0.0.2	Inferencia TFLite no dispositivo
expo-camera	17.0.0	Acesso a camera com permissoes
expo-image-picker	15.0.0	Selecao de imagem da galeria
expo-sqlite	15.0.0	Banco de dados SQLite local
axios	1.6.0	Cliente HTTP para requisicoes futuras
uuid	9.0.0	Geracao de IDs unicos

3.2 Pipeline ML - Python

Biblioteca	Versao	Funcao
TensorFlow	2.15.0	Framework de Deep Learning + conversao TFLite
Keras	Integrado	API de alto nivel para construcao do modelo
MobileNetV2	ImageNet	Backbone pre-treinado para Transfer Learning
OpenCV	4.8.1.78	CLAHE normalizacao de imagens
Pillow	10.1.0	Processamento PIL de imagens
NumPy	1.26.4	Operacoes matriciais
HuggingFace Datasets	2.18.0	Download streaming de datasets
tqdm	4.66.1	Barras de progresso

3.3 Datasets Utilizados

PlantNet 300K:	HuggingFace: mikehemberger/plantnet300K 300.000 imagens 1000+ especies
PlantVillage:	HuggingFace: ButterChicken98/plantvillage-image-text-pairs Cultivos agricolas
Max por classe:	500 imagens (configuravel via --max-per-class)
Total estimado:	100.000 - 150.000 imagens processadas

4. Pipeline de Machine Learning

4.1 Etapa 1 - Preparacao dos Dados (prepare_species.py)

O script prepare_species.py realiza o download, preprocessamento e organizacao das imagens de treinamento. Cada imagem passa por duas transformacoes criticas antes de ser salva:

CLAHE - Contrast Limited Adaptive Histogram Equalization

Tecnica de normalizacao de contraste que opera no espaco de cor LAB. Divide a imagem em blocos e equaliza o histograma de cada bloco de forma independente, com limite de amplificacao (clipLimit=3.0) para evitar amplificacao de ruido. Essencial para fotos de campo com variacao de iluminacao.

```
def apply_clahe(image_array):  
    lab = cv2.cvtColor(image_array, cv2.COLOR_RGB2LAB)  
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))  
    lab[:, :, 0] = clahe.apply(lab[:, :, 0])  
    return cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)
```

Resize com Padding Proporcional

Redimensiona a imagem preservando o aspect ratio original, adicionando padding branco para atingir o tamanho 224x224 sem distorcer as caracteristicas morfologicas da planta (folhas, frutos, caule).

```
def resize_with_padding(image, size=224):  
    ratio = min(size/w, size/h)  
    new_w, new_h = int(w*ratio), int(h*ratio)  
    resized = image.resize((new_w, new_h), Image.LANCZOS)  
    result = Image.new('RGB', (size, size), (255, 255, 255))  
    result.paste(resized, ((size-new_w)//2, (size-new_h)//2))  
    return result
```

4.2 Etapa 2 - Treinamento (train_species.py)

Arquitetura MobileNetV2 com Transfer Learning em 2 fases:

```
Input: 224 x 224 x 3 (RGB)  
|  
v  
MobileNetV2 (pre-treinado ImageNet, include_top=False)  
| [1.280 filtros de features em 7x7]  
v  
GlobalAveragePooling2D --> [1280 features]  
|  
v  
Dropout(0.5) --> regularizacao  
|  
v  
Dense(num_classes, softmax) --> probabilidades por especie
```

Fase 1 - Treinamento da Cabeça (20 epocas)

- Base MobileNetV2 congelada (pesos ImageNet fixos)
- Apenas o Dense head e treinado
- Learning rate: 0.001 | Optimizer: Adam
- Acuracia esperada: ~75%

Fase 2 - Fine-tuning (15 epocas)

- Ultimas 50 camadas do MobileNetV2 descongeladas
- Aprendizado especifico para dominio agricola
- Learning rate: 0.0001 (mais conservador)
- Acuracia esperada: ~85-90%

Augmentacao de Dados (apenas durante treinamento)

IMPORTANTE: As layers de augmentacao sao aplicadas via `dataset.map()` durante o treinamento e NAO fazem parte do modelo final. Isso garante compatibilidade 100% com TFLite TFLITE_BUILTINS.

```
get_augmentation_dataset(base_dataset):  
    RandomFlip('horizontal_and_vertical') # orientacao variada  
    RandomRotation(0.25)                 # +/- 45 graus  
    RandomZoom(0.15)                     # +/- 15% zoom  
    RandomTranslation(0.1, 0.1)          # +/- 10% posicao  
    RandomBrightness(0.2)                # +/- 20% brilho  
    RandomContrast(0.2)                  # +/- 20% contraste  
--> aplicado via dataset.map() APENAS em fit()
```

4.3 Etapa 3 - Conversao TFLite

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)  
converter.optimizations = [tf.lite.Optimize.DEFAULT] # INT8  
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS]  
tflite_model = converter.convert()  
# Resultado: model_species.tflite (~12 MB)
```

4.4 Estimativas de Tempo e Recursos

CPU (recomendado):	8 cores @ 3GHz 8-16 GB RAM ~2-3 horas
GPU (opcional):	NVIDIA GTX 1650+ ~45 minutos
Disco necessario:	50 GB (datasets + modelos intermediarios)
Tamanho final:	model_species.tflite: ~12 MB labels.json: ~5 KB

5. Estrutura do Frontend

5.1 Arvore de Arquivos

```
app_tc/  
  src/  
    App.tsx                # Root com providers  
    screens/  
      HomeScreen.tsx       # Menu principal  
      CameraScreen.tsx     # Captura de foto  
      ProcessingScreen.tsx  # Loading de analise  
      ResultScreen.tsx     # Resultado da identificacao  
      InstitutionalScreen.tsx # Biblioteca agronomica  
      CategoryScreen.tsx   # Categorias de conteudo  
      ArticleScreen.tsx    # Visualizacao de artigo  
    services/  
      api/analysisService.ts # Identificacao de especies  
      api/client.ts          # Axios HTTP client  
      api/institutionalService.ts  
      storage/database.ts    # SQLite operations  
      camera/cameraService.ts # Camera + Galeria  
      ml/tfliteService.ts    # Inferencia TFLite  
      sync/autoSyncService.ts # Sync automatico  
    hooks/  
      useAnalysis.ts        # Orquestrador de analise  
      useCamera.ts          # Camera state  
      useHistory.ts         # Historico  
      useTFLiteAnalysis.ts  # Hook TFLite  
    context/  
      AnalysisContext.tsx   # Estado global de analise  
      AppContext.tsx       # Estado global app  
    models/                 # TypeScript interfaces  
    navigation/  
      RootNavigator.tsx    # Bottom tabs + stacks  
    utils/constants.ts      # Cores, spacing, UI  
    styles/theme.ts  
    assets/models/  
      model_species.tflite  # Modelo quantizado  
      labels_species.json   # Mapeamento de classes
```

5.2 Telas do Aplicativo (7 Telas)

HomeScreen:	Menu principal com card de analise de especies
CameraScreen:	Interface camera/galeria com preview da foto capturada
ProcessingScreen:	Tela de carregamento durante inferencia TFLite
ResultScreen:	Exibe especie identificada, confianca (%), descricao e recomendacoes

InstitutionalScreen:	Hub de conhecimento agronomico
CategoryScreen:	Navegacao de artigos por categoria
ArticleScreen:	Leitura completa de artigo agronomico

5.3 Navegacao

Bottom Tab Navigator (2 abas)

```
|
+-- Aba 'Analise' (icone planta)
|   AnalysisNavigator (Stack)
|       Home -> Camera -> Processing -> Result
|
+-- Aba 'Biblioteca' (icone livro)
|   InstitutionalNavigator (Stack)
|       InstitutionalHome -> Category -> Article
```

6. Banco de Dados Local (SQLite)

O aplicativo utiliza SQLite via expo-sqlite para armazenar o historico de analises localmente no dispositivo. Nao ha sincronizacao com servidor externo.

6.1 Schema da Tabela

```
CREATE TABLE analyses (  
  id          TEXT PRIMARY KEY,  
  type        TEXT,          -- 'species'  
  result      TEXT,          -- nome da especie identificada  
  confidence  REAL,          -- 0.0 a 1.0  
  description TEXT,          -- descricao da especie  
  recommendations TEXT,      -- JSON array de recomendacoes  
  imageUrl    TEXT,          -- caminho da foto capturada  
  timestamp   TEXT,          -- ISO8601  
  processingTime INTEGER,    -- milissegundos  
  createdAt   TEXT,  
  updatedAt   TEXT  
);  
  
CREATE INDEX idx_analyses_type      ON analyses(type);  
CREATE INDEX idx_analyses_timestamp ON analyses(timestamp);
```

6.2 Operacoes Disponiveis

initialize():	Cria tabelas e indices na inicializacao do app
saveAnalysis(analysis):	Insere novo registro de analise
getAllAnalyses(limit, offset):	Lista com paginacao
getAnalysesByType(type, limit):	Filtra por tipo
getAnalysisById(id):	Busca registro especifico
deleteAnalysis(id):	Remove registro individual
clearAllAnalyses():	Limpa todo o historico
getAnalysisCount():	Total de analises realizadas

7. Banco de Dados de Especies (INFO_DB)

O tfliteService.ts contem um banco de dados embutido com informacoes agronomicas completas em portugues para 30+ especies de cultivo agricola. Cada entrada contem nome popular, descricao e 6+ recomendacoes de cultivo.

7.1 Lista de Especies Cadastradas

Nome Popular	Nome Cientifico	Categoria
Tomate	<i>Solanum lycopersicum</i>	Hortalica
Batata	<i>Solanum tuberosum</i>	Tuberculo
Mandioca	<i>Manihot esculenta</i>	Raiz
Banana	<i>Musa paradisiaca</i>	Fruta Tropical
Soja	<i>Glycine max</i>	Graos
Milho	<i>Zea mays</i>	Cereal
Cana-de-Acucar	<i>Saccharum officinarum</i>	Industrial
Trigo	<i>Triticum aestivum</i>	Cereal
Cafe Arabica	<i>Coffea arabica</i>	Bebida
Cacau	<i>Theobroma cacao</i>	Fruta/Industria
Laranja	<i>Citrus sinensis</i>	Citrus
Abacate	<i>Persea americana</i>	Fruta
Coco	<i>Cocos nucifera</i>	Palmeira
Abacaxi	<i>Ananas comosus</i>	Fruta Tropical
Acerola	<i>Malpighia emarginata</i>	Fruta
Maracuja	<i>Passiflora edulis</i>	Fruta Tropical
Goiaba	<i>Psidium guajava</i>	Fruta
Mamoa	<i>Carica papaya</i>	Fruta Tropical
Pimentao	<i>Capsicum annuum</i>	Hortalica
Repolho	<i>Brassica oleracea</i>	Hortalica
Alface	<i>Lactuca sativa</i>	Hortalica
Cenoura	<i>Daucus carota</i>	Raiz
Cebola	<i>Allium cepa</i>	Bulbo
Salsa	<i>Petroselinum crispum</i>	Erva
Manjericao	<i>Ocimum basilicum</i>	Erva Aromatica

7.2 Estrutura de Dados por Especie

```
INFO_DB[key] = {
  name:           string,    // 'Tomate'
  description:    string,    // descricao em portugues
  recommendations: string[],  // 6+ dicas de cultivo
}
```

```
}
```

Exemplo (Tomate):

```
name: 'Tomate'
description: 'O tomate (Solanum lycopersicum) e uma das
             hortalias mais cultivadas no Brasil...'
recommendations: [
  'Sol pleno, minimo 6-8 horas diarias'
  'Irigacao regular, evitar encharcamento'
  'Tutoreamento necessario para variedades altas'
  'Adubacao rica em fosforo e potassio'
  'Colheita 70-90 dias apos o transplante'
  ...
]
```

8. Intervencao Cientifica - Grad-CAM (XAI)

PROPOSTA DE INTERVENCAO CIENTIFICA PARA TCC

8.1 Conceito: Explainable AI com Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) e uma tecnica de Inteligencia Artificial Explicavel (XAI) proposta por Selvaraju et al. (2017) que gera mapas de calor visuais indicando QUAIS regioes da imagem foram determinantes para a classificacao da rede neural.

No contexto do AgroVision, Grad-CAM permite demonstrar que o modelo foca nas estruturas morfologicas relevantes da planta (folhas, caule, frutos) e nao em elementos do fundo da imagem (solo, ceu, sombras).

8.2 Hipotese Cientifica

H0: Um modelo MobileNetV2 treinado para identificacao de especies agricolas ativa regioes morfologicamente relevantes da planta como foco primario de classificacao (validado via Grad-CAM).

H1: O preprocessamento CLAHE direciona a atencao do modelo para estruturas da planta, reduzindo ativacoes em regioes de fundo em comparacao com imagens sem preprocessamento.

8.3 Como Funciona (Formula)

O Grad-CAM computa gradientes da saida da classe predita em relacao ao ultimo mapa de features convolucional:

1. Forward pass: imagem -> modelo -> probabilidade da classe c
2. Backprop: $d(y_c) / d(A^k)$ [gradiente em relacao ao feature map k]
3. Peso de importancia: $\alpha_k = \text{GlobalAvgPool}(\text{gradientes do mapa } k)$
4. Mapa de ativacao: $L_{\text{Grad-CAM}} = \text{ReLU}(\sum(\alpha_k * A^k))$
5. Upsample para tamanho original (224x224)
6. Sobrepor heatmap colorido sobre imagem original

8.4 Implementacao no AgroVision (Python)

```
import tensorflow as tf
import numpy as np
import cv2

def compute_gradcam(model, image_array, class_idx):
    # Modelo intermediario ate ultima camada conv
    last_conv = model.get_layer('Conv_1') # MobileNetV2
    grad_model = tf.keras.Model(
        inputs=model.input,
        outputs=[last_conv.output, model.output]
    )
    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(image_array)
```

```
loss = predictions[:, class_idx]
grads = tape.gradient(loss, conv_outputs)
pooled = tf.reduce_mean(grads, axis=(0, 1, 2))
cam = np.dot(conv_outputs[0], pooled)
cam = np.maximum(cam, 0) # ReLU
cam = cv2.resize(cam, (224, 224))
cam = (cam - cam.min()) / (cam.max() - cam.min())
return cam # Heatmap normalizado 0-1
```

8.5 Experimento Proposto

- Passo 1: Selecionar 50 imagens (2 por especie) para conjunto de teste
- Passo 2: Executar predicao com modelo treinado
- Passo 3: Gerar Grad-CAM para cada imagem corretamente classificada
- Passo 4: Avaliar: heatmap cobre a planta (>60%) ou o fundo (<40%)?
- Passo 5: Comparar Grad-CAM de imagens com e sem CLAHE
- Passo 6: Calcular IoU entre heatmap e mascara manual da planta
- Passo 7: Concluir: o modelo aprende features corretas?

8.6 Metricas de Avaliacao

Localization Accuracy: % de heatmaps centrados na regio da planta

IoU (Intersection over Union): Sobreposicao heatmap vs mascara manual

Background Noise Ratio: % de ativacao em regioes de fundo

Confidence vs Localization: Correlacao entre confianca e precisao do heatmap

CLAHE Impact: Diferenca de IoU com/sem preprocessamento

8.7 Valor Cientifico e Academico

- XAI e area de pesquisa ativa e relevante na literatura (2017-2025)
- Resultado visual e demonstravel na defesa do TCC
- Prova que o modelo nao e uma 'caixa preta': e interpretavel
- Valida o impacto do CLAHE no foco do modelo
- Aumenta confianca do usuario agricola no sistema
- Referencia citavel: Selvaraju et al. (2017) - Grad-CAM: Visual Explanations

9. Roadmap do Projeto

9.1 Fase 1 - Concluido (v1.0)

- [OK] Arquitetura base React Native / Expo configurada
- [OK] Navegacao com React Navigation (tabs + stacks)
- [OK] 7 telas implementadas com design system
- [OK] Servicos modulares: Camera, Storage, API, ML
- [OK] Banco SQLite com historico de analises
- [OK] Aba Institucional com conteudo agronomico
- [OK] Banco de dados de 25+ especies em portugues

9.2 Fase 2 - Concluido (v1.5 - Refatoracao ML)

- [OK] Identificacao do problema: augmentacao incompativel com TFLite
- [OK] Refatoracao completa do train_species.py
- [OK] Augmentacao movida para fora do modelo (dataset.map)
- [OK] Conversao TFLite funcional sem erros
- [OK] Quantizacao INT8 implementada (~12 MB)
- [OK] prepare_species.py com CLAHE + Padding Proporcional
- [OK] Suporte a PlantNet300k + PlantVillage (HuggingFace)
- [OK] 500 imagens por classe configurado

9.3 Fase 3 - Em Andamento (v2.0)

- [] Download e preparacao dos datasets (prepare_species.py)
- [] Treinamento do modelo final (train_species.py)
- [] Deploy do modelo treinado na pasta assets/models/
- [] Teste end-to-end no app com modelo real
- [] Implementacao do Grad-CAM (intervencao TCC)

9.4 Fase 4 - Planejado (v2.5 - TCC Final)

- [] Implementacao completa do Grad-CAM no pipeline Python
- [] Integracao Grad-CAM no ResultScreen (heatmap visual)
- [] Experimento cientifico: 50 imagens com avaliacao IoU
- [] Documentacao e escrita do TCC
- [] Preparacao para defesa

9.5 Fase 5 - Futuro (pos-TCC)

- [] Active Learning: fine-tuning automatico com fotos do usuario
- [] Deteccao de doencas (dataset adicional)
- [] Identificacao de pragas

- [] Suporte a mais de 100 especies
- [] Deploy na Play Store / App Store

10. Guia de Execucao

10.1 Pipeline de Treinamento

```
# 1. Navegar para o diretorio ML
cd c:\Users\Auvo\Desktop\Luiz\agrovision-ml-service

# 2. Ativar ambiente virtual
venv_agro\Scripts\activate

# 3. Preparar datasets (30-60 min)
python prepare_species.py --max-per-class 500

# 4. Treinar modelo (1-2 horas CPU)
python train_species.py

# 5. Copiar modelo para o app
copy model_species.tflite ..\app_tc\assets\models\
copy labels_species.json ..\app_tc\assets\models\
```

10.2 Executar o App

```
# 1. Navegar para o frontend
cd c:\Users\Auvo\Desktop\Luiz\app_tc

# 2. Instalar dependencias
npm install

# 3. Iniciar servidor Expo
npx expo start

# 4. Escanear QR Code com Expo Go (Android/iOS)
# ou pressionar 'a' para emulador Android
```

10.3 Requisitos do Sistema

Node.js:	>= 18.x
Python:	>= 3.9
RAM:	8+ GB recomendado para treinamento
Disco:	50 GB livre para datasets
Expo Go:	App instalado no celular fisico ou emulador

11. Referencias Academicas

Grad-CAM

Selvaraju, R.R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV 2017.

MobileNetV2

Sandler, M. et al. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018.

Transfer Learning

Pan, S.J. & Yang, Q. (2010). A Survey on Transfer Learning. IEEE Transactions on Knowledge and Data Engineering.

CLAHE

Zuiderveld, K. (1994). Contrast Limited Adaptive Histogram Equalization. Graphics Gems IV, Academic Press.

PlantNet

Garcin, C. et al. (2021). Pl@ntNet-300K: a plant image dataset with high label ambiguity and a long-tailed distribution. NeurIPS 2021.

TensorFlow Lite

Google. (2023). TensorFlow Lite: On-Device ML for Mobile and Embedded Devices. tensorflow.org/lite

React Native

Meta. (2023). React Native: Learn once, write anywhere. reactnative.dev

XAI Survey

Arrieta, A.B. et al. (2020). Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges. Information Fusion.